	Politechnika Bydgoska im. J. J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki		
Przedmiot	Zaawansowane programowanie obiektowe		
Prowadzący	dr inż. Damian Szczegielniak		
Temat	Aplikacja monolityczna - połączenie usługi typu REST oraz aplikacji webowej		
Student	Jakub Ziółkowski, grupa 1, Informatyka stosowana, IV semestr, studia stacjonarne		
Nr ćw.	6	Data wykonania	11.07.2026 r.
Ocena		Data oddania spr.	02.08.2026 r.

Spis treści

1. Cel laboratorium.....	2
2. Rozwiązanie	2
2.1. Architektura monolityczna zapewnia integrację warstw i eliminuje narzut sieciowy	2
2.2. Konfiguracja projektu Gradle i pliku application.properties	4
2.3. Refaktoryzacja kontrolerów i unifikacja modelu danych.....	5
2.4. Konfiguracja modułu Spring Security	7
2.5. Testowanie i weryfikacja aplikacji monolitycznej.....	8
1. Interfejs Webowy Thymeleaf:	8
2. Dokumentacja Swagger UI i REST API:	9
3. Automatyczne testy JUnit:.....	10
3. Wnioski końcowe / Podsumowanie	10

1. Cel laboratorium

Celem tego laboratorium była realizacja monolitycznej wersji systemu zarządzania projektami. Wykonanie tego zadania polegało na połączeniu w jeden projekt o nazwie project-monolith dwóch wcześniej osobnych części: back-endowej usługi REST API (działającej wcześniej niezależnie na porcie 8080) oraz front-endowej aplikacji webowej MVC opartej na szablonach Thymeleaf (działającej wcześniej na porcie 8081).

Aplikacja monolityczna uruchamiana jest w pojedynczym procesie serwera Apache Tomcat na porcie 8080. Zapewnia jednocześnie działanie usług: dostarczania widoków HTML dla użytkowników przeglądarki oraz udostępniania punktów końcowych REST API dla klientów zewnętrznych i Swagger UI.

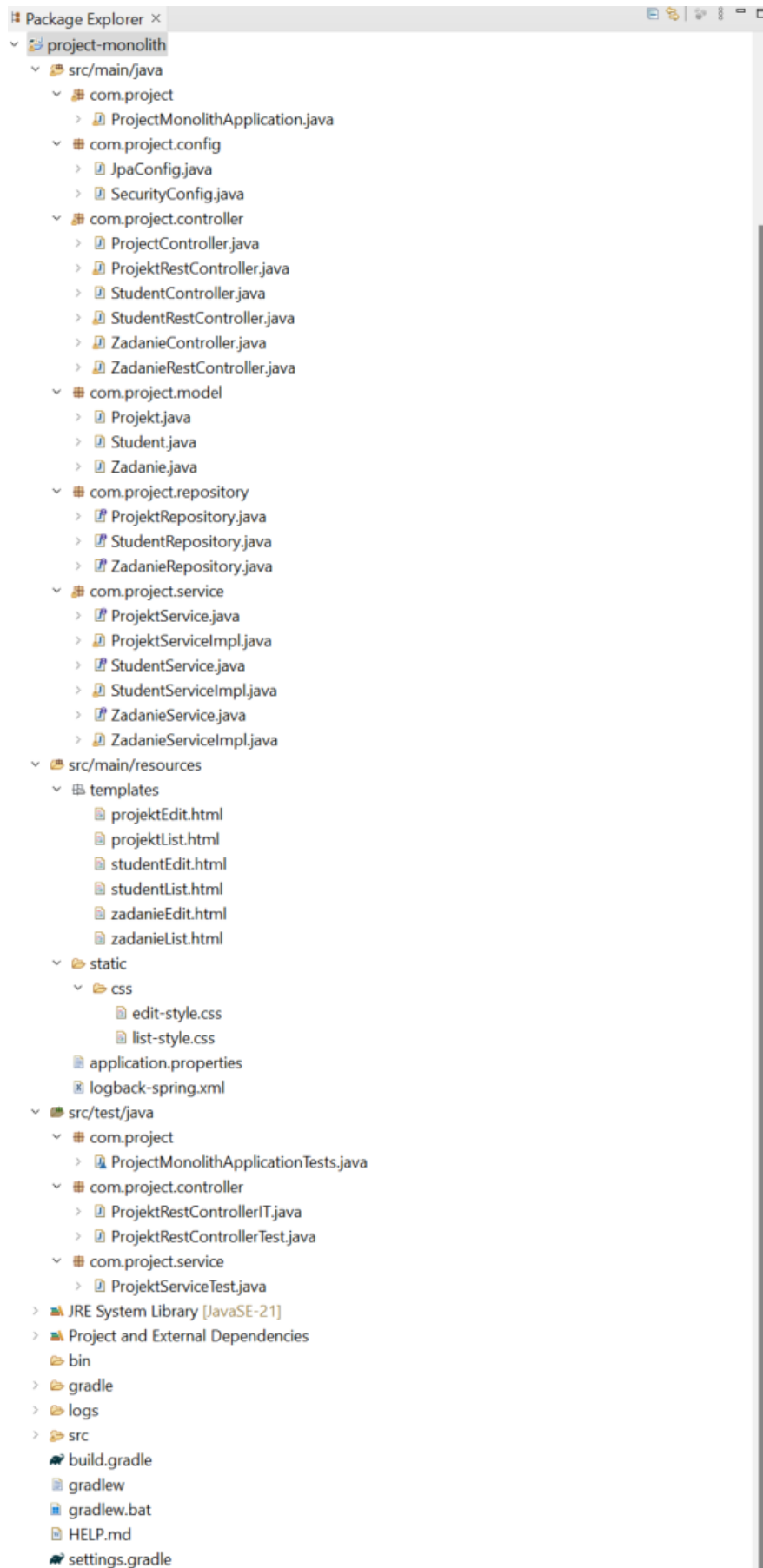
Głównym założeniem było zastąpienie narzutu sieciowego HTTP spowodowanego obiektem RestClient uruchamianiem metod serwisów domenowych w pamięci aplikacji, zachowując dotychczasową funkcjonalność obu warstw.

2. Rozwiązanie

2.1. Architektura monolityczna zapewnia integrację warstw i eliminuje narzut sieciowy

W podejściu zaprezentowanym podczas laboratoriów 4 i 5 komunikacja zachodziła poprzez sieć: aplikacja webowa przekazywała zapytania użytkownika, generując żądania HTTP za pomocą obiektu RestClient do serwera REST API.

W aplikacji typu monolit obie warstwy współdzielą jeden kontekst aplikacji Spring Boot. Kontrolery MVC (ProjectController, StudentController, ZadanieController) nie wysyłają żądań sieciowych, ale wywołują metody serwisów domenowych (ProjektService, StudentService, ZadanieService), które wchodzi w interakcję z repozytoriami Spring Data JPA i bazą danych PostgreSQL. Równolegle w tej samej aplikacji funkcjonują kontrolery REST (np. ProjektRestController), obsługujące żądania pod ścieżkami /api/...



Screen 1 - Widok struktury pakietów projektu project-monolith w oknie Package Explorer środowiska Eclipse IDE

2.2. Konfiguracja projektu Gradle i pliku application.properties

Projekt project-monolith powstał na bazie skopiowanego i zmodyfikowanego projektu project-rest-api. W pliku settings.gradle zdefiniowałem nazwę projektu: rootProject.name = 'project-monolith'.

Plik build.gradle łączy zależności wymagane przez back-end JPA/REST oraz front-end Thymeleaf:

```
build.gradle x
1 plugins {
2   id 'java'
3   id 'org.springframework.boot' version '4.1.0'
4   id 'io.spring.dependency-management' version '1.1.7'
5 }
6
7 group = 'com.project'
8 version = '0.0.1-SNAPSHOT'
9
10 java {
11   toolchain {
12     languageVersion = JavaLanguageVersion.of(21)
13   }
14 }
15
16 repositories {
17   mavenCentral()
18 }
19
20 dependencies {
21   implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
22   implementation 'org.springframework.boot:spring-boot-starter-security'
23   implementation 'org.springframework.boot:spring-boot-starter-validation'
24   implementation 'org.springframework.boot:spring-boot-starter-webmvc'
25   implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:3.0.2'
26   implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
27   compileOnly 'org.projectlombok:lombok'
28   developmentOnly 'org.springframework.boot:spring-boot-devtools'
29   runtimeOnly 'org.postgresql:postgresql'
30   annotationProcessor 'org.projectlombok:lombok'
31   testImplementation 'org.springframework.boot:spring-boot-starter-data-jpa-test'
32   testImplementation 'org.springframework.boot:spring-boot-starter-security-test'
33   testImplementation 'org.springframework.boot:spring-boot-starter-validation-test'
34   testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
35   testCompileOnly 'org.projectlombok:lombok'
36   testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
37   testAnnotationProcessor 'org.projectlombok:lombok'
38 }
39
40 compileJava {
41   options.encoding = 'UTF-8'
42 }
43
44 compileTestJava {
45   options.encoding = 'UTF-8'
46 }
47
48 tasks.named('test') {
49   useJUnitPlatform()
50   testLogging {
51     showStandardStreams = true //ustawia drukowanie komunikatów w konsoli
52   }
53 }
54
```

Screen 2 - Zawartość pliku build.gradle w projekcie project-monolith

Plik `src/main/resources/application.properties` zawiera konfigurację uruchomieniową na porcie 8080, bez zbędnego parametru `rest.base.url` z projektu poprzedniego laboratorium:

```
application.properties ×
1 spring.application.name=project-monolith
2
3 # Spring DataSource
4 spring.datasource.url=jdbc:postgresql://localhost:5432/projekty
5 spring.datasource.username=postgres
6 spring.datasource.password=postgres
7 spring.datasource.driver-class-name=org.postgresql.Driver
8
9 # Spring JPA / Hibernate
10 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
11 spring.jpa.hibernate.ddl-auto=update
12 spring.jpa.show-sql=true
13 spring.jpa.properties.hibernate.format_sql=true
14
15 # Spring Security credentials
16 spring.security.user.name=admin
17 spring.security.user.password=admin
18
19 spring.threads.virtual.enabled=true
```

Screen 3 - Zawartość pliku `application.properties` w projekcie `project-monolith`

2.3. Refaktoryzacja kontrolerów i unifikacja modelu danych

Klasy encyjne JPA (Projekt, Student, Zadanie) są dostępne wprost w tym samym projekcie, dlatego nie występuje potrzeba stosowania klas DTO i pomocniczej klasy `RestResponsePage`. Nie występują również klasy `RestClient` i `ServiceUtil`. Kontrolery MVC działają bezpośrednio na obiektach encyjnych.

W kodzie kontrolerów wyeliminowałem przechwytywanie wyjątku `HttpStatusCodeException` (który występuje tylko w przypadku komunikacji sieciowej). Metody domenowych serwisów są wywoływane w pamięci aplikacji. Kontrolery komunikują się z interfejsami `ProjektService`, `StudentService` oraz `ZadanieService`.

```

1 package com.project.controller;
2
3 import jakarta.validation.Valid;
4 import java.util.HashSet;
5 import java.util.Set;
6 import org.springframework.data.domain.Pageable;
7 import org.springframework.stereotype.Controller;
8 import org.springframework.ui.Model;
9 import org.springframework.validation.BindingResult;
10 import org.springframework.web.bind.annotation.GetMapping;
11 import org.springframework.web.bind.annotation.ModelAttribute;
12 import org.springframework.web.bind.annotation.PostMapping;
13 import org.springframework.web.bind.annotation.RequestParam;
14 import com.project.model.Projekt;
15 import com.project.model.Student;
16 import com.project.service.ProjektService;
17 import com.project.service.StudentService;
18
19 @Controller
20 public class ProjectController {
21
22     private final ProjektService projektService;
23     private final StudentService studentService;
24
25     public ProjectController(PjektService projektService, StudentService studentService) {
26         this.projektService = projektService;
27         this.studentService = studentService;
28     }
29
30     @GetMapping("/projektList")
31     public String projektList(Model model, Pageable pageable) {
32         model.addAttribute("projekty", projektService.getProjekty(pageable).getContent());
33         return "projektList";
34     }
35
36     @GetMapping("/projektEdit")
37     public String projektEdit(@RequestParam(name="projektId", required=false) Integer projektId, Model model, Pageable pageable) {
38         if (projektId != null) {
39             model.addAttribute("projekt", projektService.getProjekt(projektId).orElse(new Projekt()));
40         } else {
41             model.addAttribute("projekt", new Projekt());
42         }
43         model.addAttribute("v_studenci", studentService.getStudenci(pageable).getContent());
44         return "projektEdit";
45     }
46
47     @PostMapping(path = "/projektEdit")
48     public String projektEditSave(@ModelAttribute("projekt") @Valid Projekt projekt,
49                                 BindingResult bindingResult,
50                                 @RequestParam(name="selectedStudentIds", required=false) Set<Integer> selectedStudentIds,
51                                 Model model, Pageable pageable) {
52         if (bindingResult.hasErrors()) {
53             model.addAttribute("v_studenci", studentService.getStudenci(pageable).getContent());
54             return "projektEdit";
55         }
56
57         if (selectedStudentIds != null && !selectedStudentIds.isEmpty()) {
58             Set<Student> studenci = new HashSet<>();
59             for (Integer studentId : selectedStudentIds) {
60                 studentService.getStudent(studentId).ifPresent(studenci::add);
61             }
62             projekt.setStudenci(studenci);
63         }
64
65         projektService.setProjekt(projekt);
66         return "redirect:/projektList";
67     }
68
69     @PostMapping(params="cancel", path = "/projektEdit")
70     public String projektEditCancel() {
71         return "redirect:/projektList";
72     }
73
74     @PostMapping(params="delete", path = "/projektEdit")
75     public String projektEditDelete(@ModelAttribute("projekt") Projekt projekt) {
76         projektService.deleteProjekt(projekt.getProjektId());
77         return "redirect:/projektList";
78     }
79 }

```

Screen 4 - Kod źródłowy klasy ProjectController.java

2.4. Konfiguracja modułu Spring Security

Aplikacja monolityczna musi zabezpieczać dwa typy ruchu: interakcję w przeglądarce dla widoków Thymeleaf oraz wywołania REST API / Swagger UI.

W klasie `SecurityConfig.java` skonfigurowałem równoległe logowanie formularzowe (`formLogin`) oraz uwierzytelnianie nagłówków Basic Authentication (`httpBasic`), zapewniając jednocześnie dostęp do statycznych zasobów CSS oraz dokumentacji OpenAPI:

```
SecurityConfig.java x
1 package com.project.config;
2
3 import org.springframework.context.annotation.Bean;
4
5 @Configuration
6 public class SecurityConfig {
7
8     @Bean
9     public SecurityFilterChain filterChain(HttpSecurity httpSecurity) throws Exception {
10         return httpSecurity
11             .csrf(csrf -> csrf.disable())
12             .authorizeHttpRequests(auth -> auth
13                 .requestMatchers("/css/**", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
14                 .anyRequest().authenticated()
15             )
16             .formLogin(Customizer.withDefaults())
17             .httpBasic(Customizer.withDefaults())
18             .build();
19     }
20 }
21
22
```

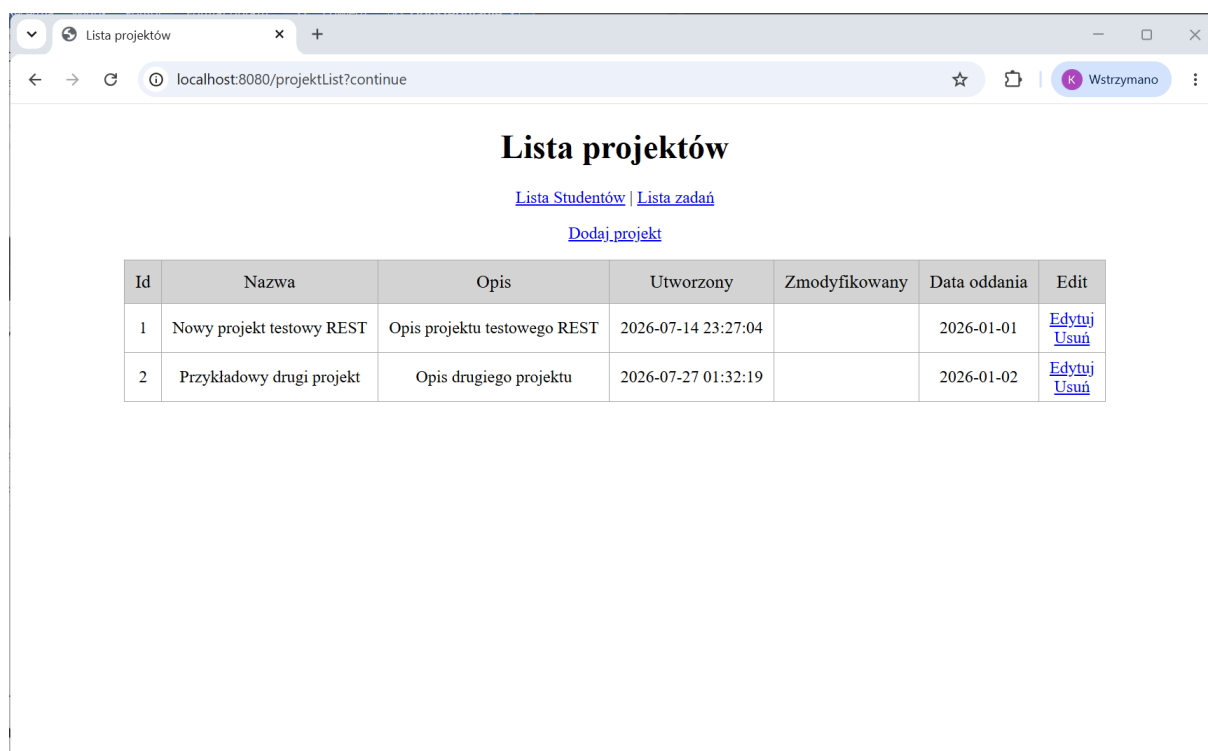
Screen 5 - Klasa `SecurityConfig.java` w projekcie monolitycznym

2.5. Testowanie i weryfikacja aplikacji monolitycznej

Po uruchomieniu klasy głównej `ProjectMonolithApplication.java` w środowisku Eclipse IDE przeprowadziłem pełną weryfikację monolitu, składającą się z części:

1. Interfejs Webowy Thymeleaf:

Po przejściu pod adres `http://localhost:8080/projektList` w przeglądarce pojawia się strona logowania Spring Security. Po zalogowaniu danymi `admin/admin` wyświetlane są widoki listy projektów, a także dostępnych pod linkami studentów oraz zadań. Wszystkie funkcjonalności: odczyt, dodawanie, edycja, usuwanie rekordów działają poprawnie, a zmiany są zapisywane w bazie danych PostgreSQL projekty.

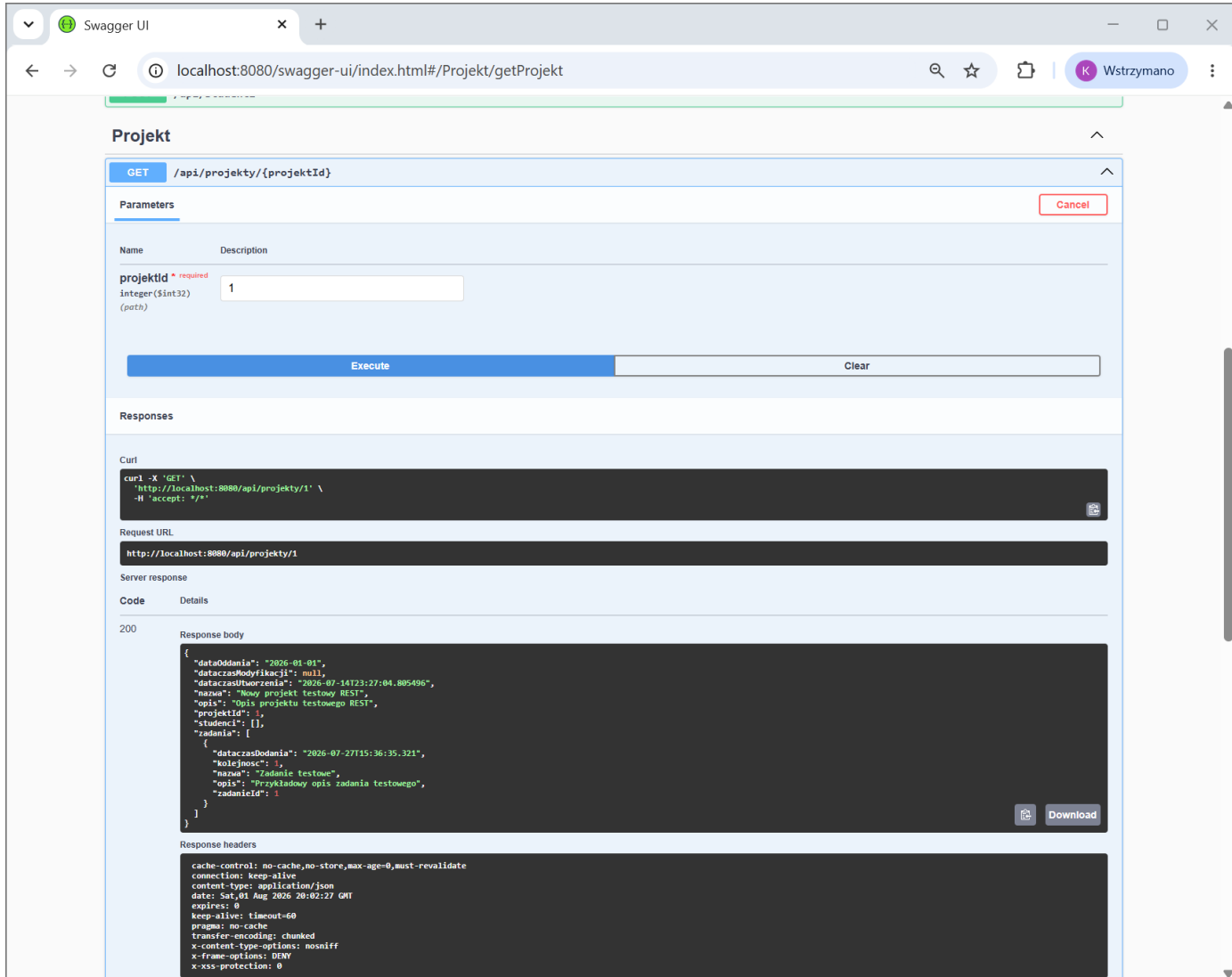


Id	Nazwa	Opis	Utworzony	Zmodyfikowany	Data oddania	Edit
1	Nowy projekt testowy REST	Opis projektu testowego REST	2026-07-14 23:27:04		2026-01-01	Edytuj Usuń
2	Przykładowy drugi projekt	Opis drugiego projektu	2026-07-27 01:32:19		2026-01-02	Edytuj Usuń

Screen 6 - Poprawny widok listy projektów pod adresem `http://localhost:8080/projektList` w przeglądarce internetowej

2. Dokumentacja Swagger UI i REST API:

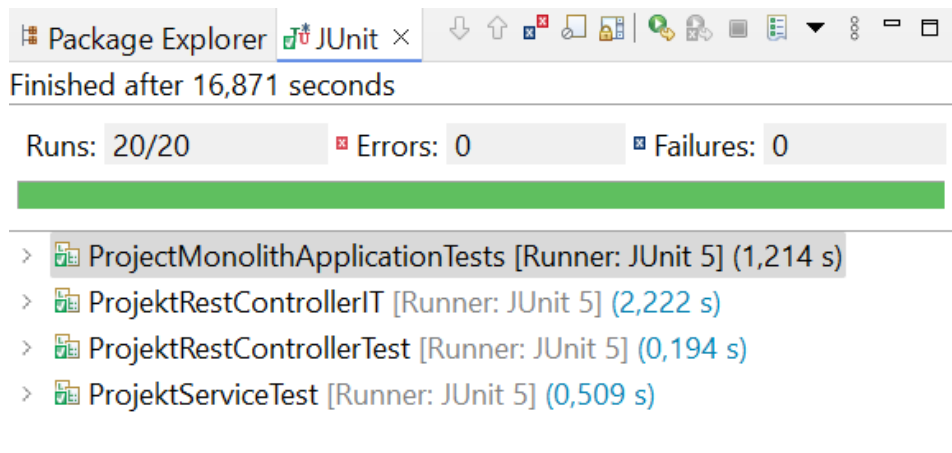
Pod adresem <http://localhost:8080/swagger-ui/index.html> dostępny jest interfejs OpenAPI z endpointami, np. `/api/projekty`. Wykonanie zapytania testowego (przycisk *Try it out*) po podaniu identyfikatora projektu zwraca dane w formacie JSON.



Screen 7 - Okno interfejsu Swagger UI przedstawiające udane wykonanie zapytania REST API pod adresem z portem 8080

3. Automatyczne testy JUnit:

Wykonałem zestaw testów jednostkowych i integracyjnych z pakietów `com.project`, `com.project.controller` oraz `com.project.service` w `src/test/java` (`ProjectMonolithApplicationTests`, `ProjektRestControllerIT`, `ProjektRestControllerTest`, `ProjektServiceTest`).



Screen 8 - Widok zakładki JUnit w Eclipse IDE - bezbłędne wykonanie wszystkich 20 testów

3. Wnioski końcowe / Podsumowanie

Sprawozdanie szóste pozwoliło wyciągnąć wnioski dotyczące różnic pomiędzy architekturą rozproszoną (REST API + osobna aplikacja webowa) a architekturą monolityczną:

- **Wydajność i eliminacja opóźnień:** Przejście na architekturę monolityczną kompletnie skreśla narzut sieciowy HTTP oraz zbędne operacje ponownej serializacji i deserializacji obiektów JSON po stronie front-endu. Wywołania metod serwisów odbywają się w pamięci aplikacji w tym samym procesie.
- **Prostota zarządzania:** Całość systemu działa jako pojedyncza usługa na jednym porcie (8080), co ułatwia lokalne zarządzanie projektem i jego uruchamianie, a także obniża wymagania sprzętowe serwera.
- **Równoległa obsługa interfejsów:** Połączenie obu projektów w jeden nie wpłynęło negatywnie na dostępność funkcjonalną. Aplikacja monolityczna równolegle i bez konfliktów obsługuje strony HTML dla użytkowników końcowych oraz udostępnia endpointy REST API dla zewnętrznych klientów.